[CDT204 – LECTURE 03]

Datapath and Pipelining

Afshin Ameri

Computer Architecture · Mälardalen University · Autumn 2026

Outline

01 • Crash course in digital electronics

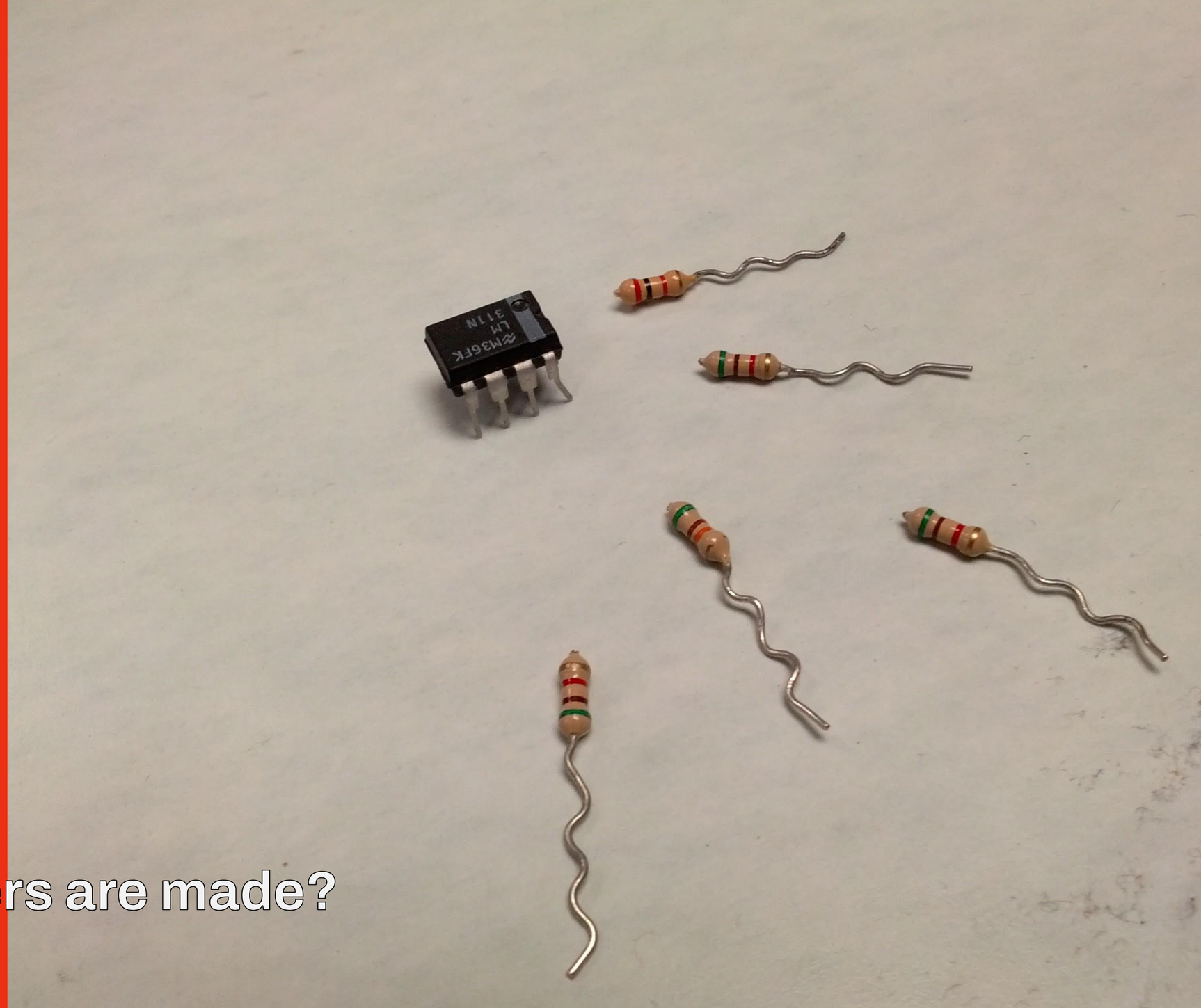
02 • Building a datapath

03 • Introduction to pipelining

04 • Pipelining hazards

Digital Electronics

Or how the computers are made?

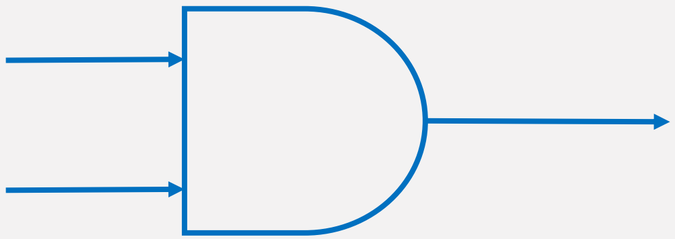


Logic gates

- Implemented by e.g. transistors
- Perform Boolean functions
  - AND
  - OR
  - XOR
  - NOT
  - ...

AND gates

- Symbol

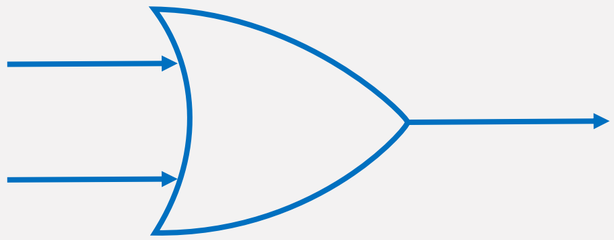


- Boolean function
  - output is 1 when both inputs are 1

Input 1 \ Input 2	0	1
0	0	0
1	0	1

OR gates

- Symbol

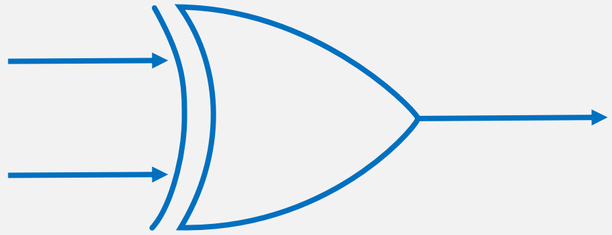


- Boolean function
  - output is 1 when either or both of the inputs are 1

Input 1 \ Input 2	0	1
0	0	1
1	1	1

XOR gates

- Symbol

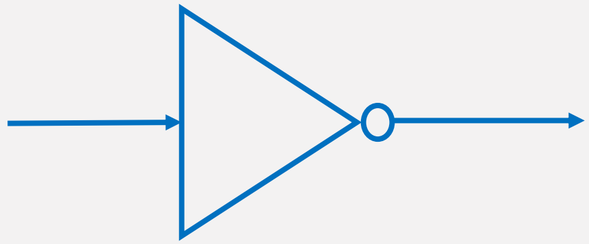


- Boolean function
  - output is 1 when either of the inputs are 1 (not both)

Input 1 \ Input 2	0	1
0	0	1
1	1	0

NOT gates

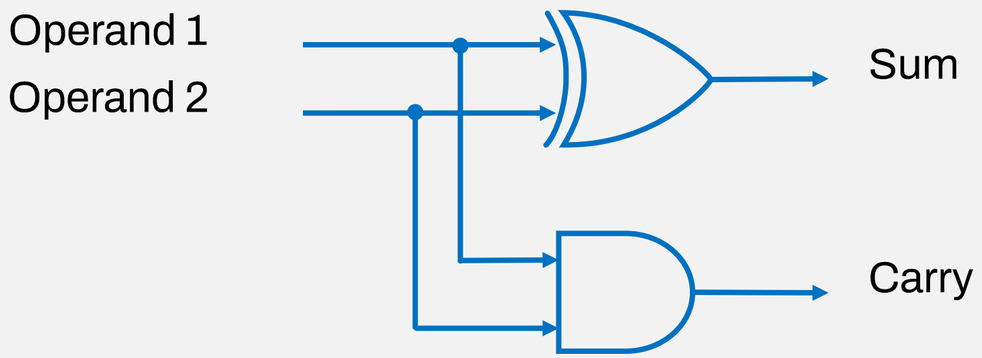
- Symbol



- Boolean function
  - output is inverse of the input

Input	
0	1
1	0

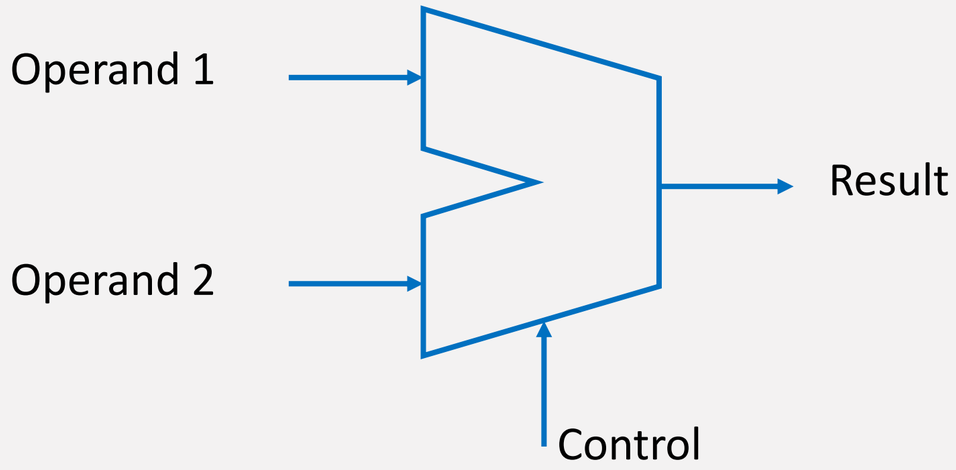
Example: Half Adder



Inputs		Outputs	
Operand 1	Operand 2	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

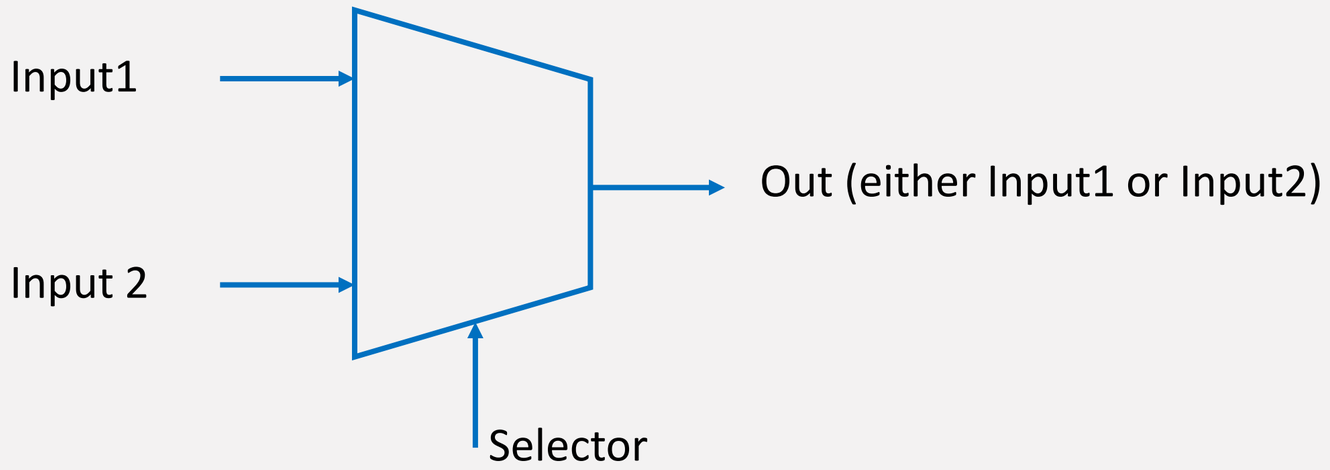
Arithmetic logic units (ALUs)

- Performs
  - Integer operations (add, sub, mul, …)
  - Bit operations (AND, OR, Shifting, …)



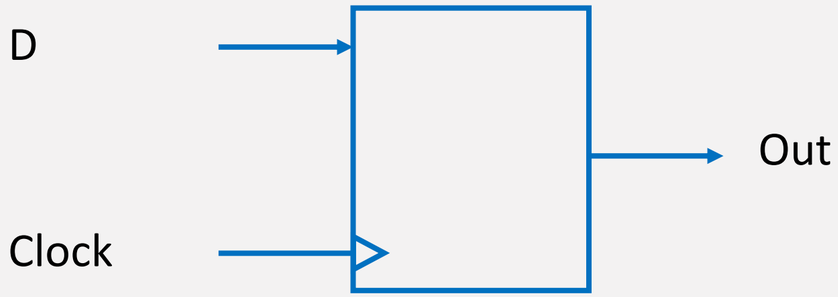
Multiplexers (MUX:s)

- Selects one of the several input signals
- Example: 2-to-1 MUX:



State Elements

- Combinational vs. sequential logic
- D flip-flop



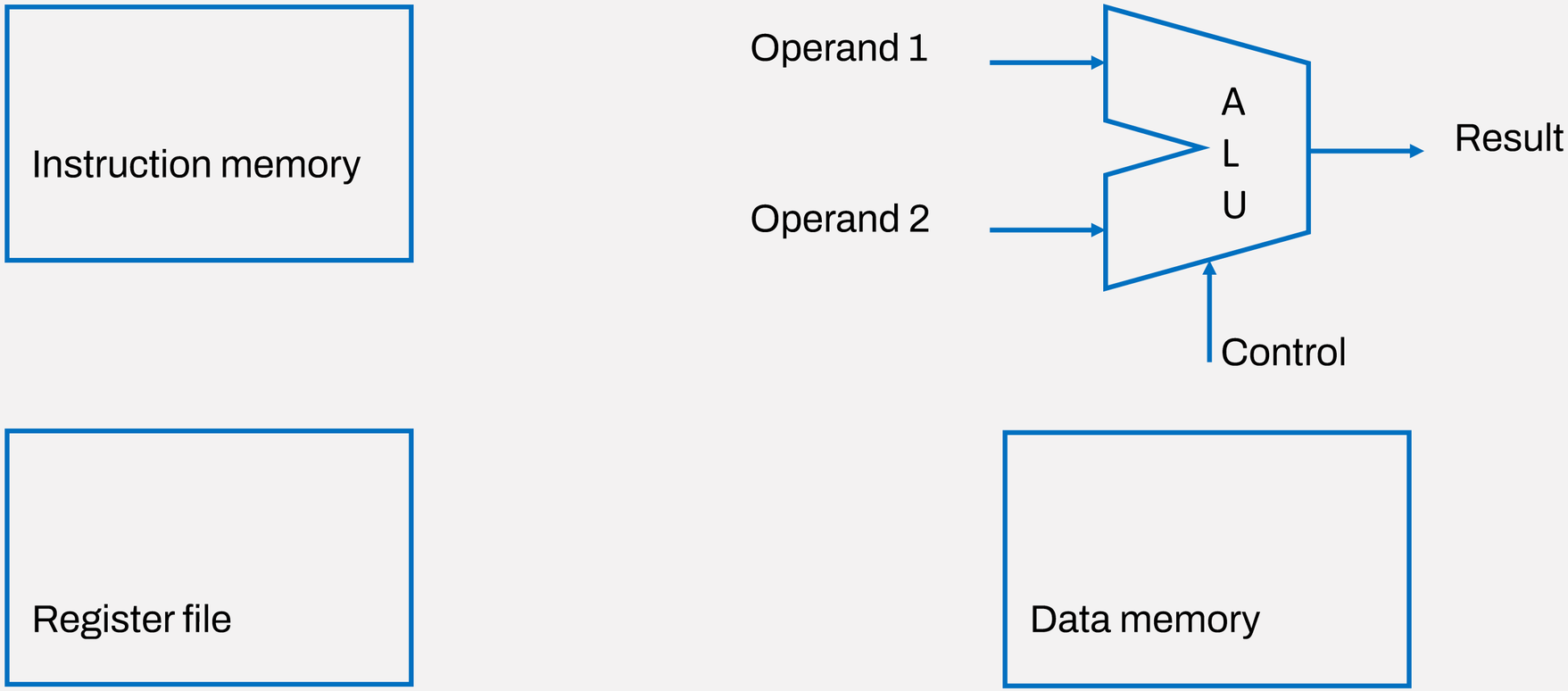


Data

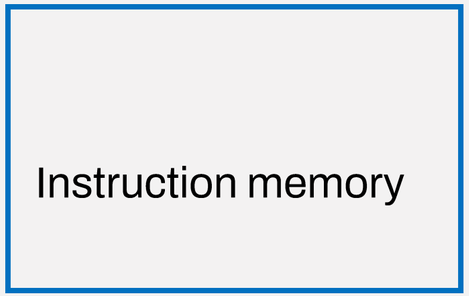
MIPS Datapath

- Fetch next instruction
  - Pointed to by program counter (PC)
- Read register file
- Depending on the instruction
  - Use ALU to calculate
    - Arithmetic result
    - Memory address fro load/store
    - Branch target address
  - Access data memory for load/store
  - PC ← target address or PC+4
  - Update register file

Hardware


Instruction memory
Operand 1
Operand 2
A
L
U
Result
Control
Register file
Data memory

Hardware



- State
- Input: PC
- Output: Instruction (32 bits)

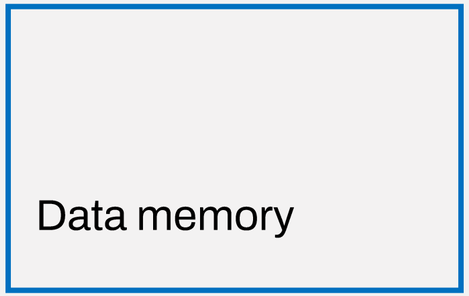
Hardware



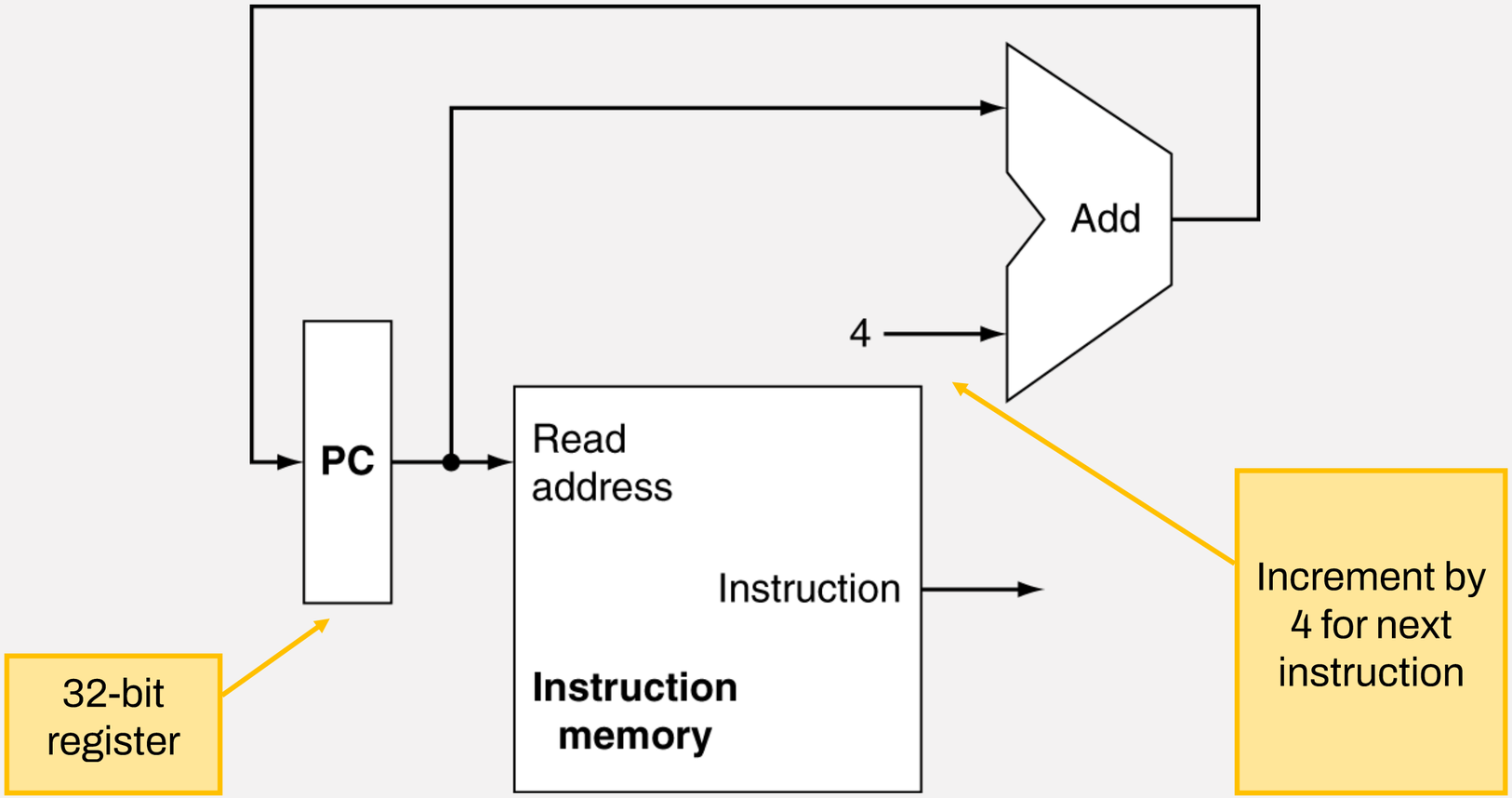
- State
- Read/write 32x32 registers

Hardware

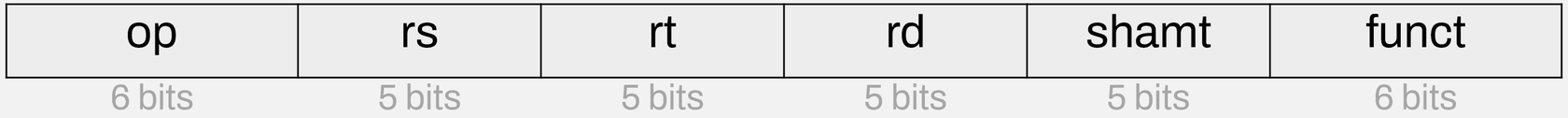
- State
- Read/write



Instruction fetch



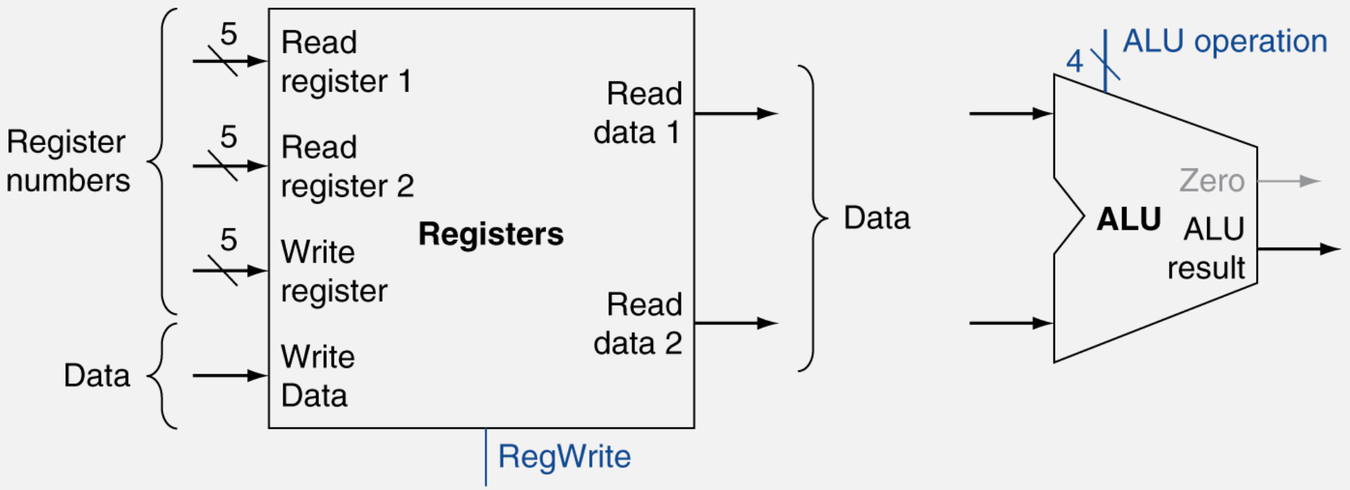
MIPS R-Format Instructions



- Instruction fields
  - **op**: operation code (opcode)
  - **rs**: first source register number
  - **rt**: second source register number
  - **rd**: destination register number
  - **shamt**: shift amount (00000 for now)
  - **funct**: function code (extends opcode)
- Used for arithmetic instructions

R-Format instructions

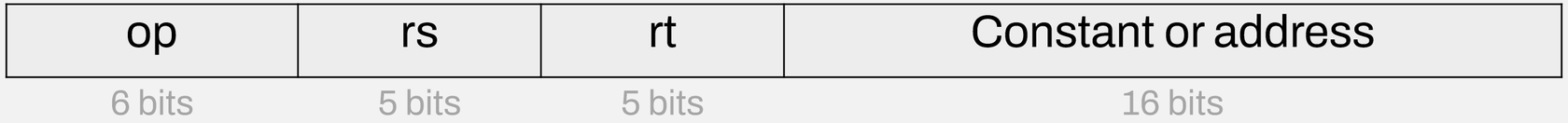
- Read two register operands
- Perform arithmetic/logical operation
- Write register result



a. Registers

b. ALU

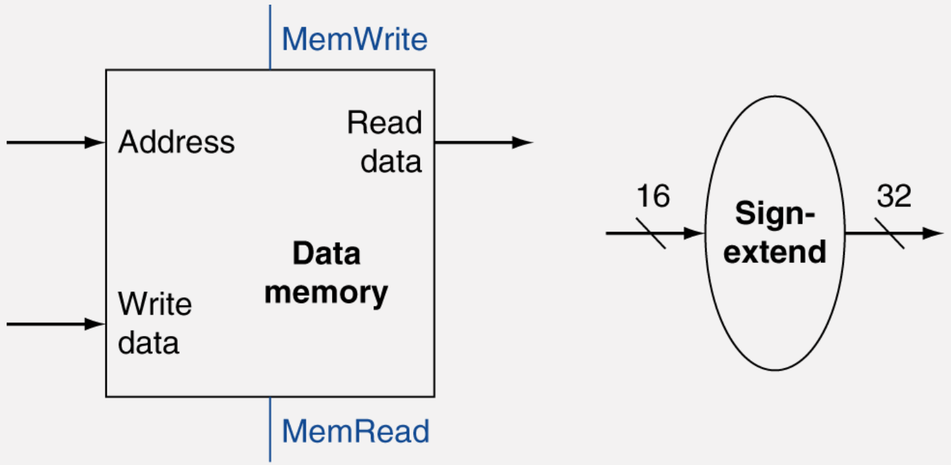
MIPS I-Format Instructions



- Instruction fields
  - **op**: operation code (opcode)
  - **rs**: source register number
  - **rt**: source/destination register number
  - **Constant or address**: signed 16-bit value
- Used for branch, transfer and immediate instructions

Load/store instructions

- Read register operands
- Calculate address using 16-bit offset
  - Use ALU, but sign extend offset
- Load: Read memory and update register
- Store: Write register value to memory



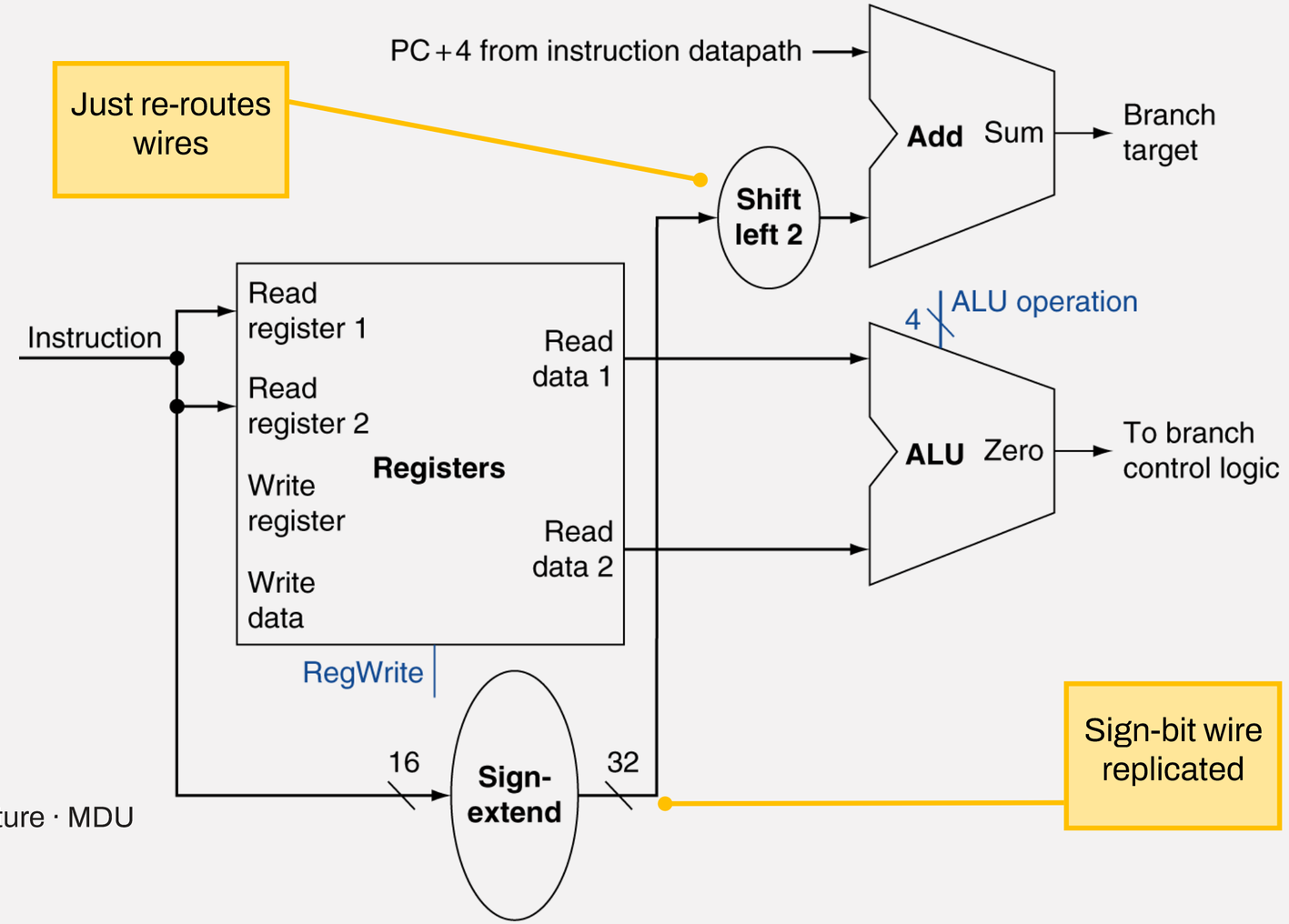
a. Data memory unit

b. Sign extension unit

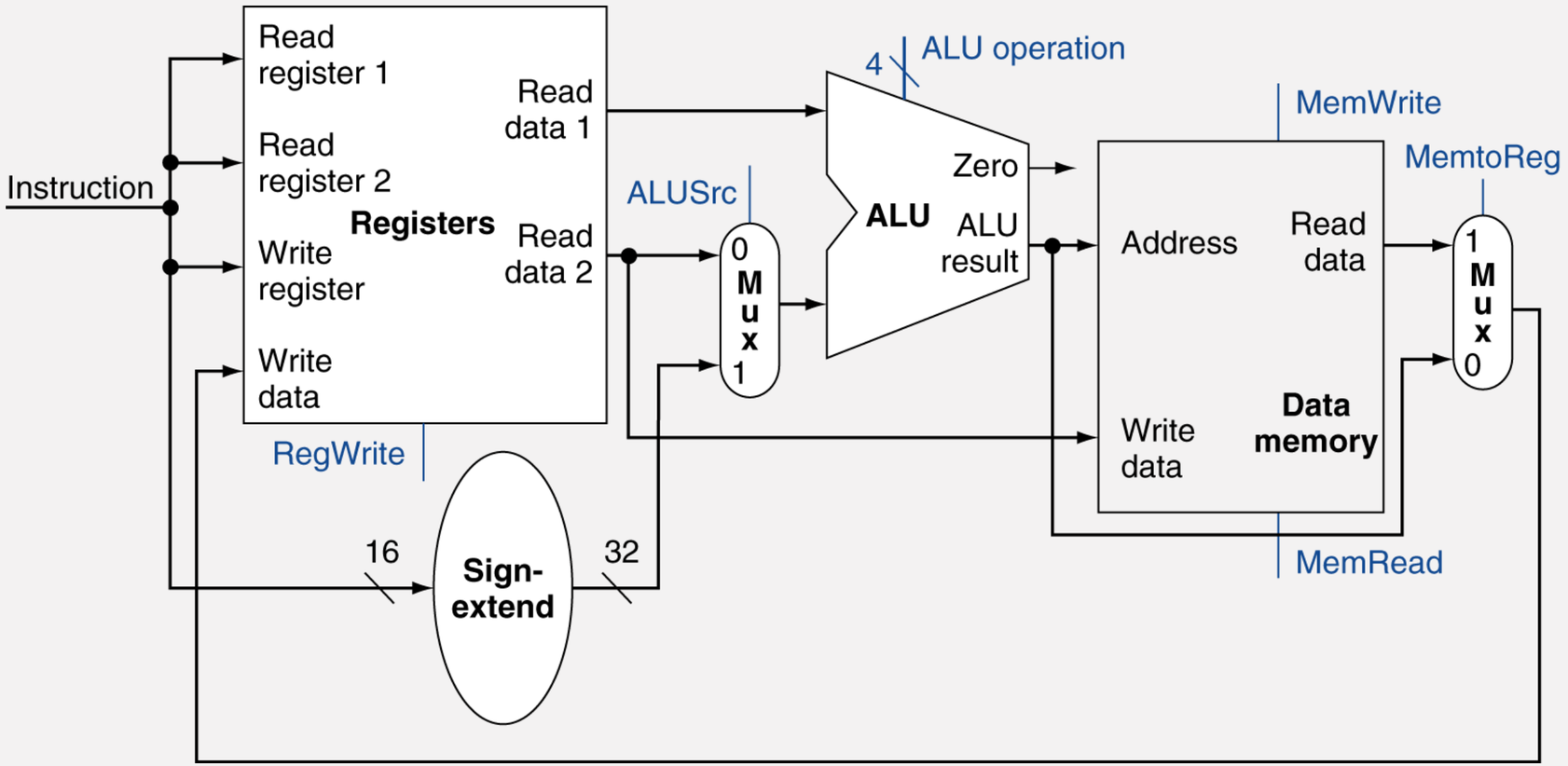
Branch instructions

- Read register operands
- Compare operands
  - Use ALU, subtract and check Zero output
- Calculate target address
  - Sign-extend displacement
  - Multiply displacement by 4
  - Add 4 to PC
    - Already calculated by instruction fetch

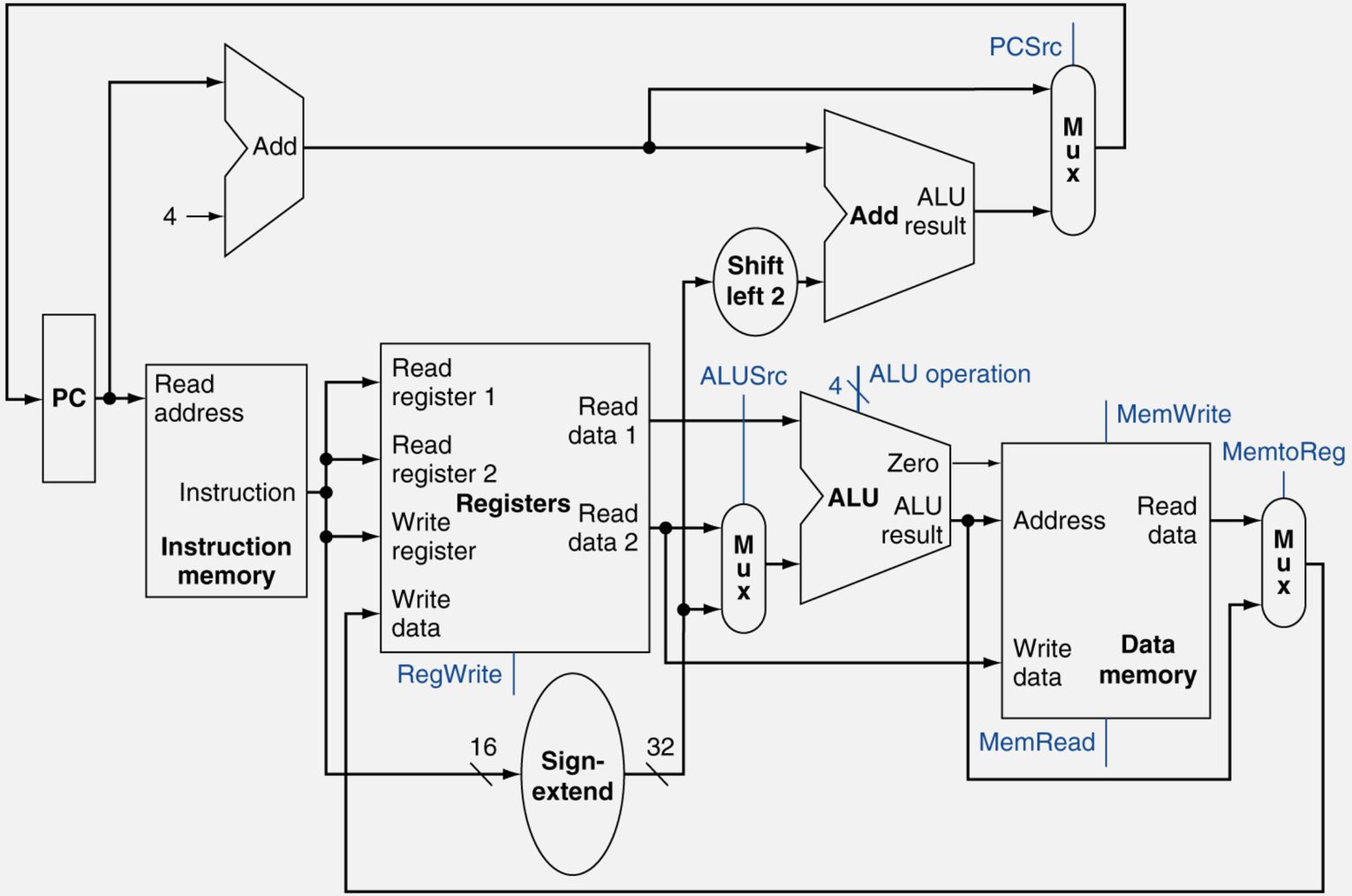
Branch instructions



R-type / Load / Store Datapath

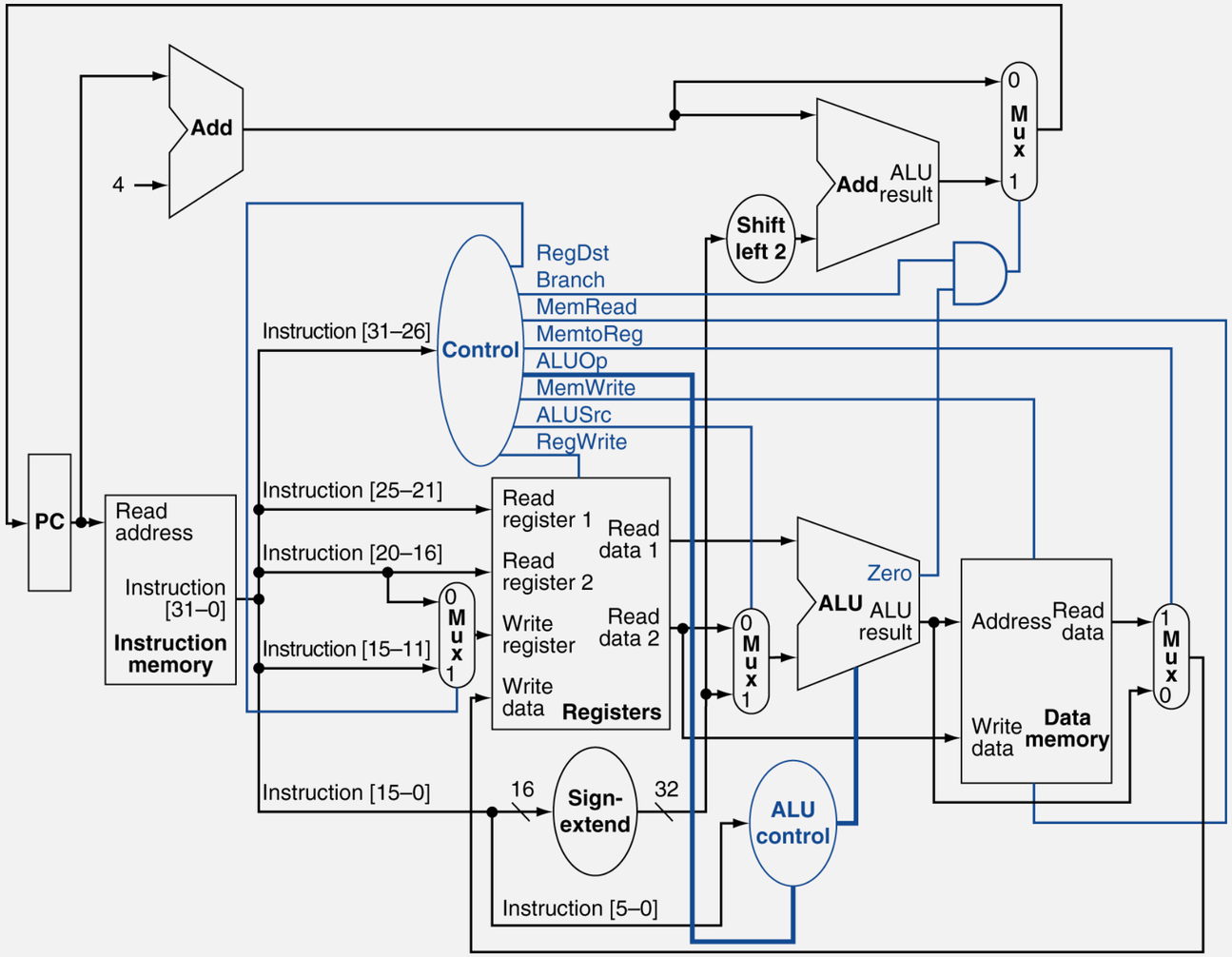


Full Datapath

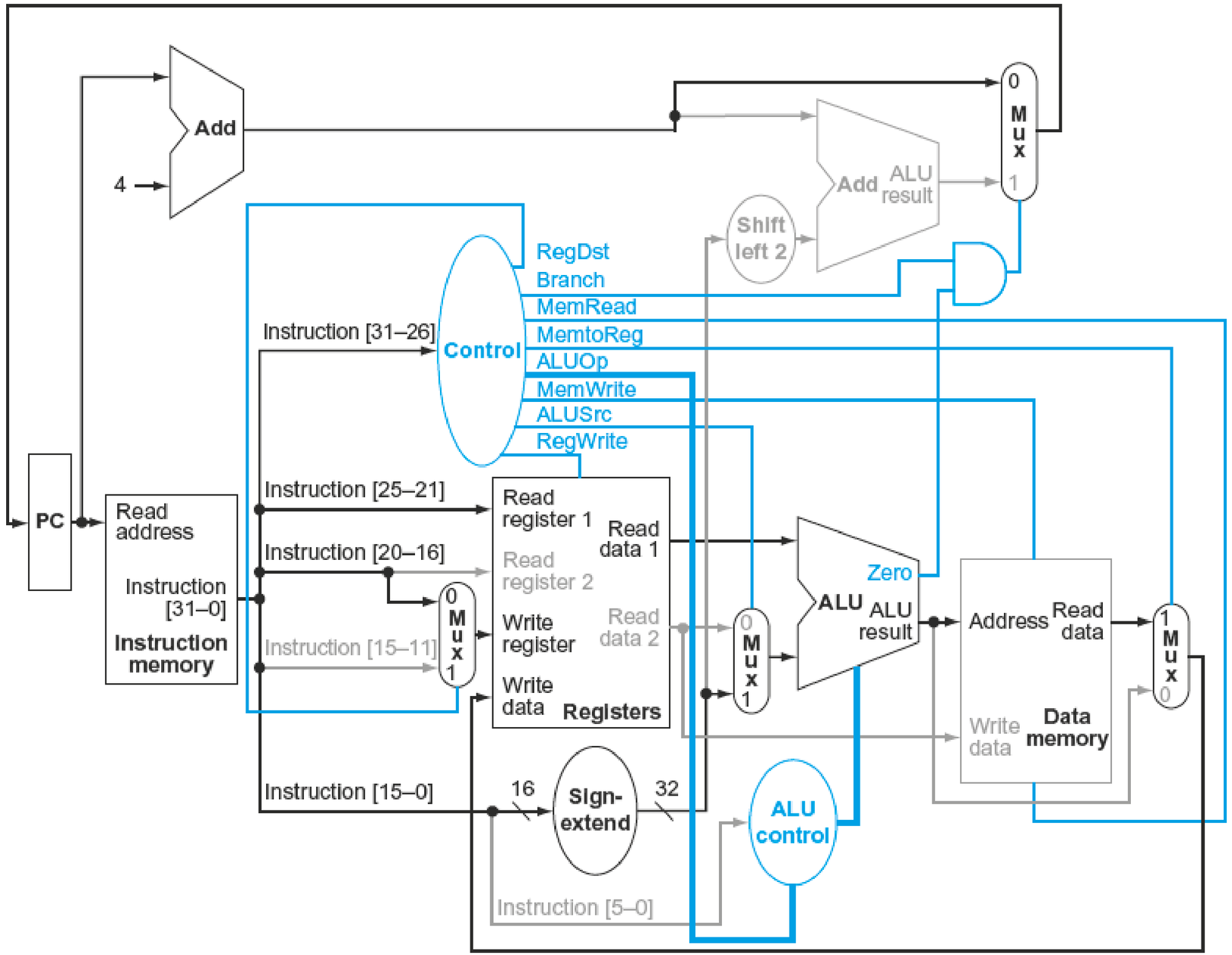


Datapath with control

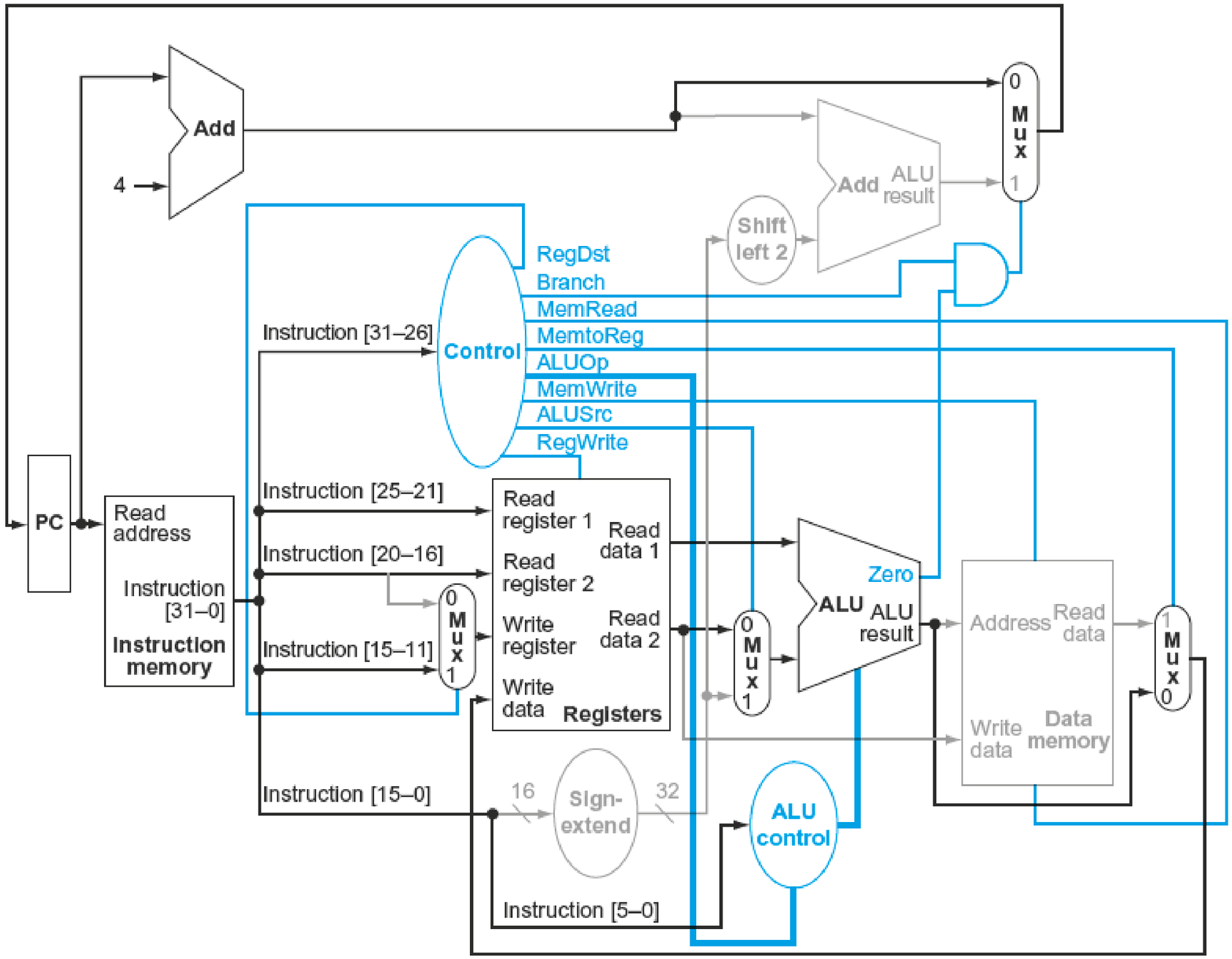
- Notice the blue (Control) lines.
- They control other parts of the Datapath.
- Based on the instruction to be executed.



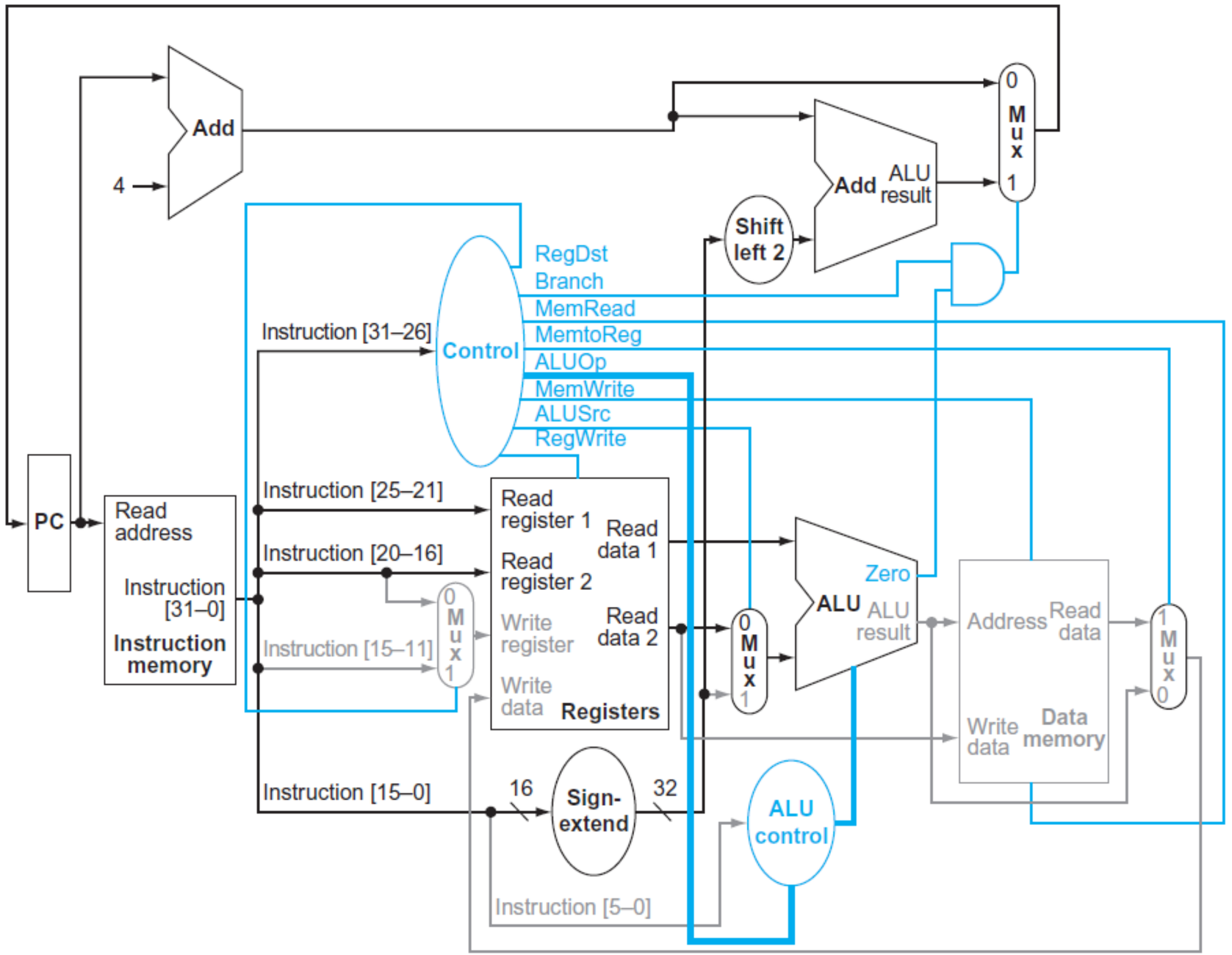
Example: Datapath for Load



Example: Datapath for R-Type instructions



Example: Datapath for BEQ





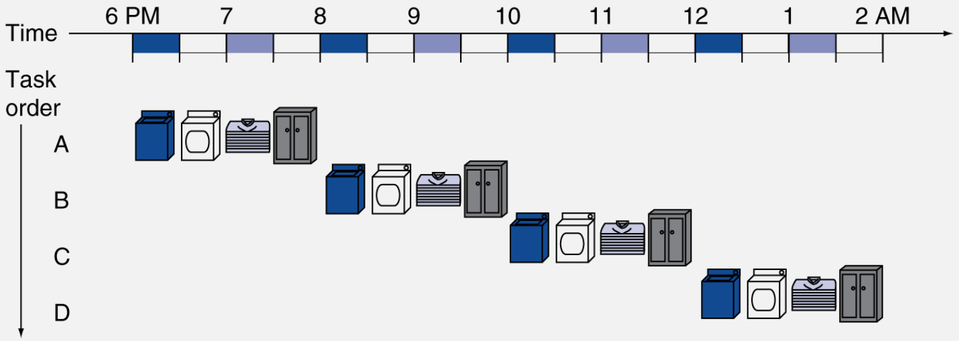
Pipelining

Performance issues

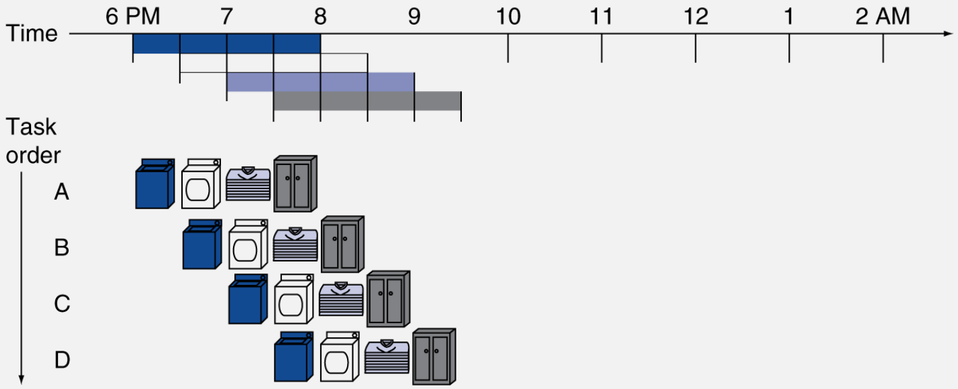
- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary clock period for different instructions
- Solution: Pipelining

Pipelined laundry

OPTION A



OPTION B



- Four loads: Speed up = 8/3.5 = 2.3
- *n* loads: Speedup = 2n/(1.5+0.5n) ≈ 4
- Equals to number of stages

Pipeline speedup

- If all stages take the same time:

$$Cycle\ time_{pipelined} = \frac{Cycle\ time_{non-pipelined}}{Number\ of\ stages}$$

-
- Speedup due to increased throughput
  - Latency (time for each instruction) does *not* decrease

MIPS pipeline

- Five stages
  - **IF**: Instruction fetch from memory
  - **ID**: Instruction decode & register read
  - **EX**: Execute operation or calculate address
  - **MEM**: Access memory operand
  - **WB**: Write result back to register

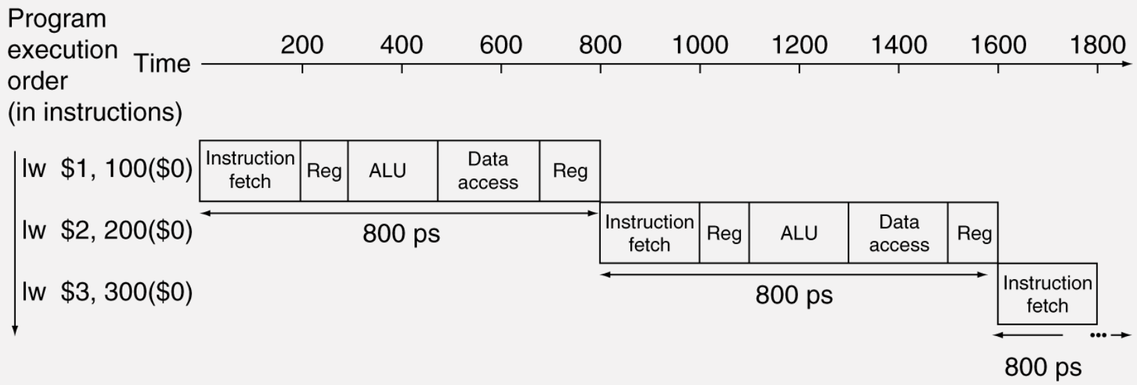
Pipeline performance

- Assume time for stage is
  - 100ps for register read or write
  - 200ps for other stages

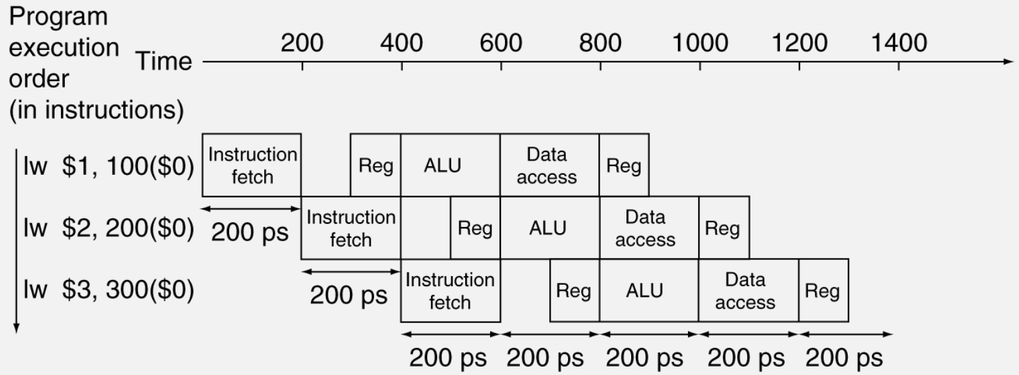
INSTRUCTION	INSTRUCTION FETCH	REGISTER READ	ALU OPERATION	MEMORY ACCESS	REGISTER WRITE	TOTAL
lw	200ps	100ps	200ps	200ps	100ps	800ps
sw	200ps	100ps	200ps	200ps	-	700ps
R-format	200ps	100ps	200ps	-	100ps	600ps
beq	200ps	100ps	200ps	-	-	500ps

Pipeline Performace

- Without pipelining



- With pipelining



MIPS ISA design for pipelining

- All instructions are 32 bits
  - Easier to fetch and decode in one cycle
  - c.f. x86: 1 to 17 byte instructions
- Fewer and regular instruction formats
  - Can decode and read registers in one step
- Load/store addressing
  - Can calculate in 3rd stage, access memory in 4th stage
- Alignment of memory operands
  - Memory access takes only one cycle

CAUTION











PIPELINE HAZARD

Pipeline hazards

- Situations that prevent starting the next instruction in the next cycle:
  - Structural hazards
    - A required resource is busy
  - Data hazards
    - Dependence on an earlier instruction’s results
  - Control hazards
    - Delay of branching decisions

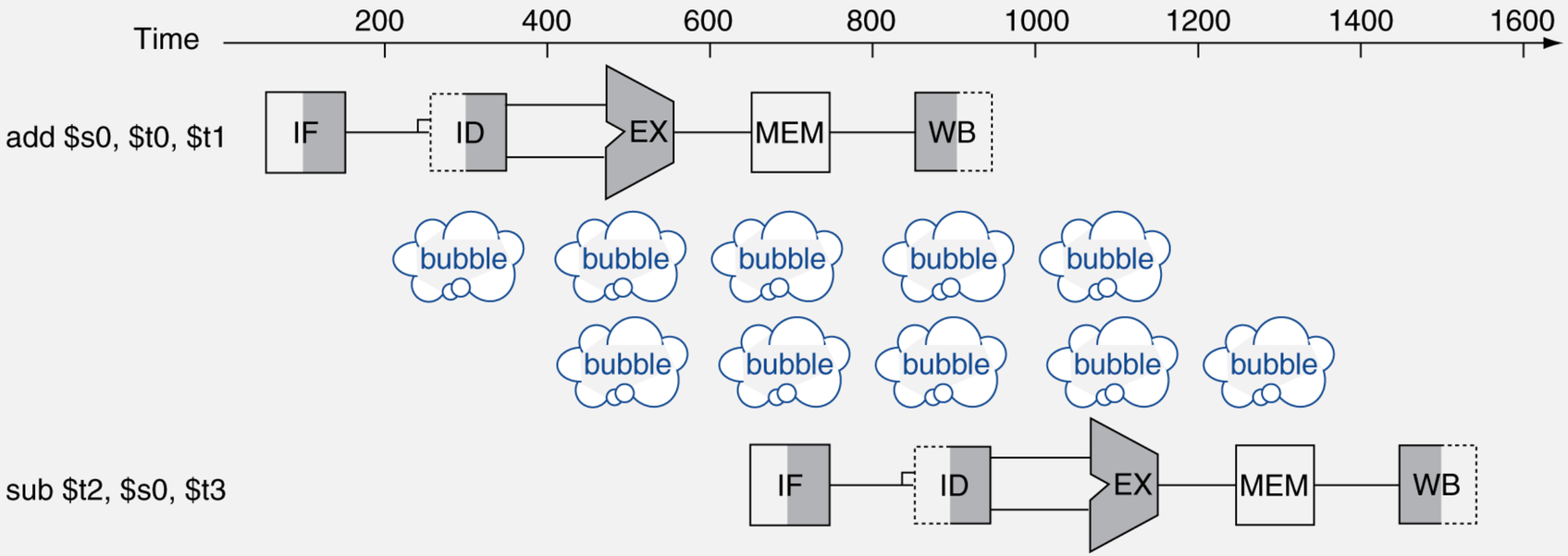
Structural hazards

- Conflict for use of a resource
- MIPS has a pipeline with a single memory
  - Load/store requires data access
  - Instruction fetch would have to stall for that cycle → would cause a pipeline *“bubble”*
- Hence, pipelined datapaths require separate instruction/data memories
  - or separate instruction/data caches

Data hazards

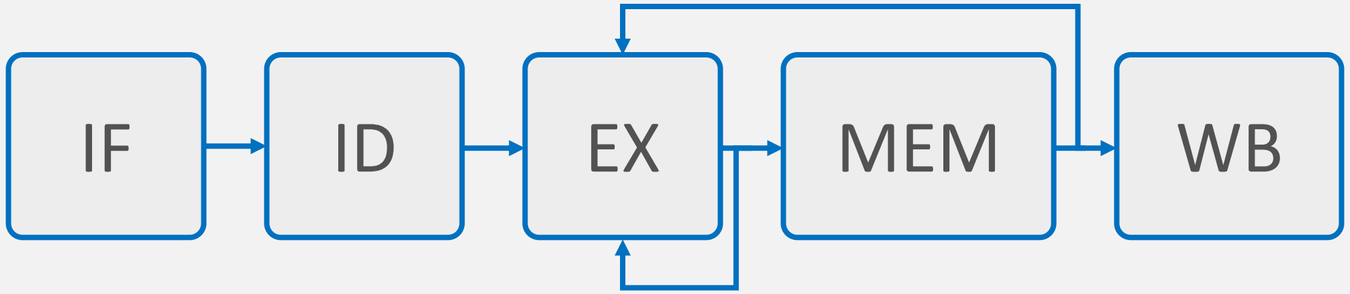
- Instruction depends on result of an earlier instruction:

```
add   $s0, $t0, $t1
sub   $t2, $s0, $t3
```

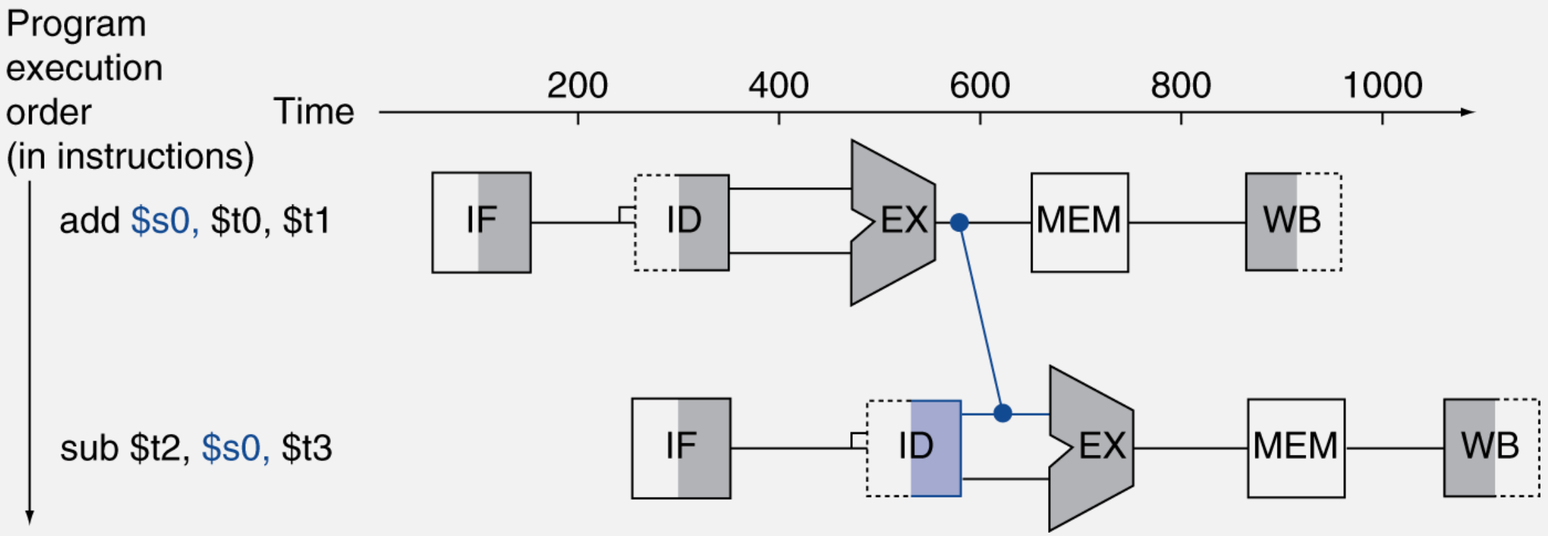


Forwarding (bypassing)

- Results immediately available via bypass connections:

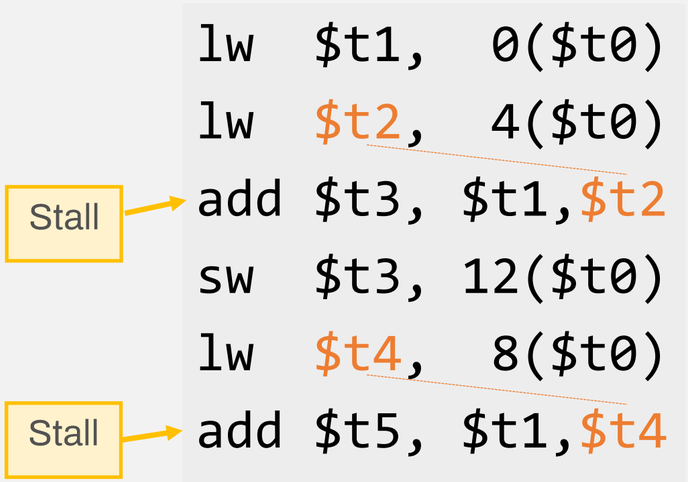


Example: Forwarding



Code scheduling

Reorder code to avoid use of load result in the next instruction




```
lw  $t1,  0($t0)
lw  $t2,  4($t0)
add $t3, $t1,$t2
sw  $t3, 12($t0)
lw  $t4,  8($t0)
add $t5, $t1,$t4
sw  $t5, 16($t0)
```

13 cycles

```
lw  $t1,  0($t0)
lw  $t2,  4($t0)
lw  $t4,  8($t0)
add $t3, $t1,$t2
sw  $t3, 12($t0)
add $t5, $t1,$t4
sw  $t5, 16($t0)
```

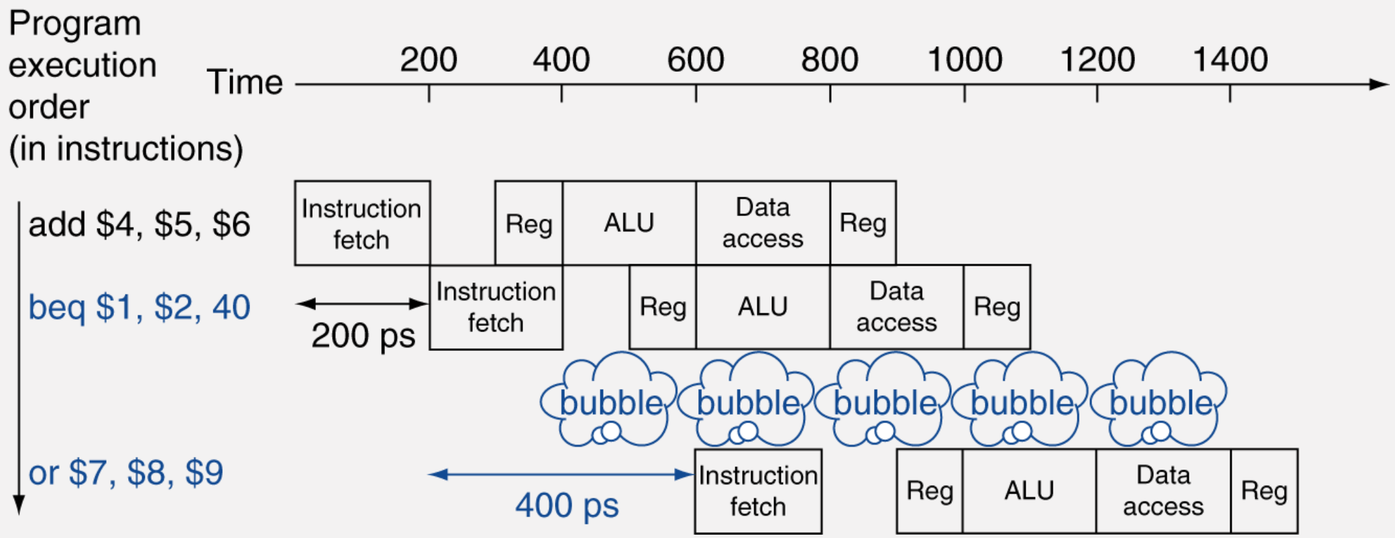
11 cycles

Branch hazards

- Branch determines flow of control
  - Next instruction to fetch depends on branch outcome
  - Pipeline can’t always fetch correct instruction
    - Still working on ID stage of branch
- In MIPS pipeline:
  - Need to compare registers and compute target early in the pipeline
  - Add hardware to do it in ID stage

Stall on branch

- Wait until branch outcome determined before fetching next instruction

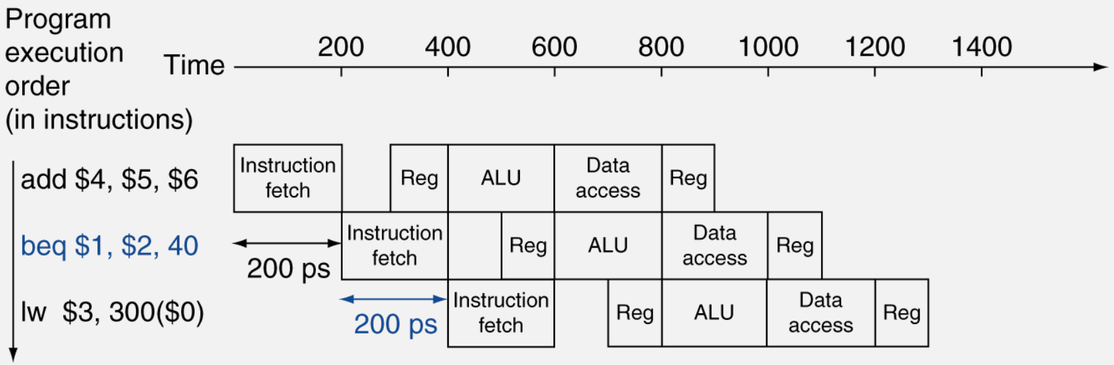


Branch prediction

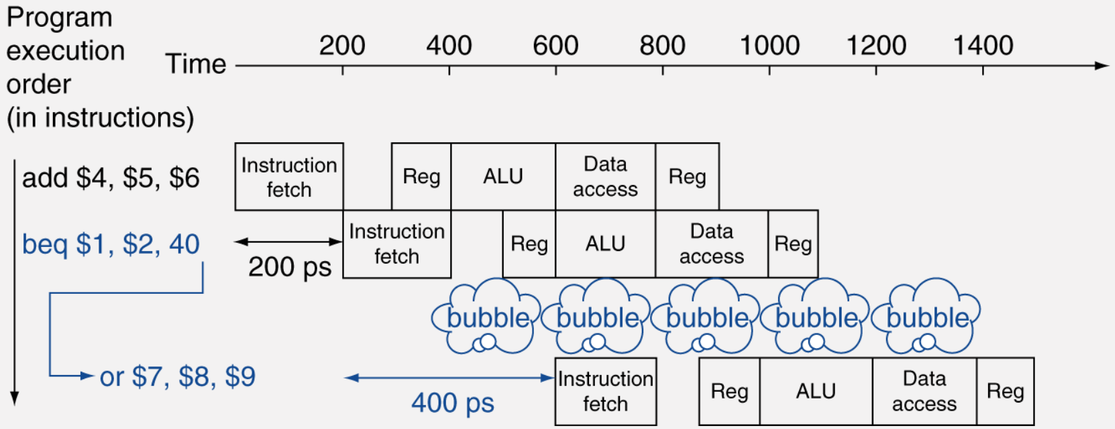
- Longer pipelines can’t readily determine branch outcome early
  - Stall penalty becomes unacceptable
  - Branch instructions are common
- Predict outcome of branch
  - Only stall if prediction is wrong
- In MIPS pipeline:
  - Can predict branches not taken
  - Fetch instruction after branch, with no delay

Example: Predict not taken

Prediction correct



Prediction incorrect



More sophisticated branch prediction

- Static branch prediction
  - Based on typical branch behavior
  - Example: loop and if-statement branches
    - Predict backward branches taken
    - Predict forward branches not taken

- Dynamic branch prediction
  - Hardware measures actual branch behavior
    - e.g. record recent history of each branch
  - Assume future behavior will continue the trend
    - When wrong, stall while re-fetching, and update history

Summary

01
- Pipelining improves performance by increasing instruction throughput: keeps latency intact, parallelizes instruction execution.

02
- Subject to hazards: Structural, Data, Control

03
- Instruction set design affects complexity of pipeline implementation

Do These

- Read chapter up to and including 4.5
- Yes! We skip chapter 3
- Continue assignment 1
- Choose your writing team and title

Next time

- Pipelined MIPS datapath
- Dynamic branch prediction
- Multiple issue
- Dynamic pipeline scheduling

HARD WORK BEATS TALENT, WHEN TALENT DOESN'T WORK HARD

TIM NOTKE